Evolutionary Computation Assignment 9: Hybrid Evolutionary Algorithm

BENGRICHE Tidiane (ER-2037)
Szczublińska Joanna Wiktoria (156070)

Contents

1 Introduction

The aim of this report is to investigate the performance of a hybrid evolutionary algorithm and to compare it with previously implemented methods such as MSLS, ILS, and LNS.

Hybrid evolutionary algorithms combine the global exploration capabilities of evolutionary strategies with local improvement to provide a balance between exploration and exploitation.

In this work, we implement a **Hybrid Evolutionary Algorithm** with a population of 20 solutions, each generated randomly and improved using a local search (edges). Parents are selected uniformly from the population, and two crossover operators are applied with equal probability:

- **Operator 1:** preserves all nodes and edges common to both parents, forming shared subpaths, and completes the solution randomly to reach 50% of nodes. The resulting subpaths are then randomly connected.
- **Operator 2:** selects one parent as the base solution, removes nodes not present in the other parent and repairs the solution using weighted k-regret heuristic (k=2 and weight = 0.5). This operator preserves not only common subpaths but also the order and traversal direction from the chosen parent.

Local search (edges) is applied to all offspring, except in a variant where it is skipped after crossover, while it is always applied to the initial population. Mechanisms are included to prevent duplicate solutions and avoid premature convergence, maintaining diversity in the population.

You can see the repository of our assignment on github:
`https://github.com/tbengric/EC_laboratory`

2 Pseudocode

In this section, we will present the pseudocodes of the algorithm. For this assignment, we will code in C++.

Structure of HEA Result

HEAResult:
best_path
local_search_count
main_loop_iterations

Hybrid Evolutionary Algorithm (HEA)

FUNCTION HEA(dist, nodes, pop_size, avg_msls_time, timer, useLocalSearch):
population ← ∅
localSearchCount ← 0
// **Initialization**
WHILE $|population| < pop\_size$ **DO**:
randPath ← RANDOM_SOLUTION(SELECT_NODES(nodes))
improved ← STEEPEST_LOCAL_SEARCH(randPath, dist, "edge", nodes)
localSearchCount ← localSearchCount + 1
INSERT_TO_POPULATION(population, improved, dist, nodes, pop_size)
iterations ← 0
// **Main loop**
WHILE timer < avg_msls_time **DO**:
iterations ← iterations + 1
(p1, p2) ← SELECT_PARENTS(population)
offspring ← APPLY randomly OPERATOR_1 or OPERATOR_2
IF useLocalSearch **THEN**:
offspring ← STEEPEST_LOCAL_SEARCH(offspring, dist, "edge", nodes)
localSearchCount ← localSearchCount + 1
INSERT_TO_POPULATION(population, offspring, dist, nodes, pop_size)
best ← individual with minimum objective value in population
RETURN HEAResult(best, localSearchCount, iterations)

INSERT_TO_POPULATION

FUNCTION INSERT_TO_POPULATION(population, solution, dist, nodes, MAX_POP_SIZE):
    score_new ← COMPUTE_OBJECTIVE(solution, dist, nodes)
    FOR each individual IN population DO:
      IF individual == solution **THEN RETURN** FALSE
      IF COMPUTE_OBJECTIVE(individual, dist, nodes) == score_new **THEN RETURN** FALSE
    IF $|population| < MAX\_POP\_SIZE$ **THEN**:
      ADD solution TO population
      RETURN TRUE
    worst ← individual with MAX objective value in population
    IF score_new < objective(worst) **THEN**:
      REPLACE worst WITH solution
      RETURN TRUE
    RETURN FALSE

SELECT_PARENTS

FUNCTION SELECT_PARENTS(population):
    p1 ← RANDOM individual from population
    p2 ← RANDOM individual from population, p2 ≠ p1
    RETURN (p1, p2)

OPERATOR_1 (Common Node/Edge Recombination)

FUNCTION OPERATOR_1(parent1, parent2, nodes):
// **Identify common nodes and edges**
    common_nodes ← $\{v \mid v \in parent1 \wedge v \in parent2\}$
    common_edges ← edges appearing in both parents (any direction)
// **Build subpaths from common edges**
    subpaths ← BUILD_SUBPATHS(common_edges)
    FOR each node IN common_nodes not in subpaths DO:
      ADD single-node subpath
    remaining ← nodes not used so far
    SHUFFLE remaining
    ADD random single-node subpaths until $len(subpaths) = len(nodes)/2$
    SHUFFLE subpaths
    offspring ← CONCATENATE subpaths in random orientation
    RETURN offspring

OPERATOR_2 (Intersection + Repair)

FUNCTION OPERATOR_2(parent1, parent2, nodes, dist):
base ← RANDOM choice of parent1 or parent2
other ← the remaining parent
offspring ← nodes of base that also appear in other
offspring ← WEIGHTED_K_REGRET(offspring, dist, nodes)
RETURN offspring

OPERATOR_1 (Common Node/Edge Recombination)

FUNCTION OPERATOR_1(parent1, parent2, nodes):
p1 ← copy of parent1
p2 ← copy of parent2
// Step 1: Identify common nodes and edges
in_p1 ← set of nodes in p1
in_p2 ← set of nodes in p2
common_nodes ← nodes present in both in_p1 and in_p2
common_edges ← empty list
FOR each consecutive pair (u,v) in p1 DO
IF edge (u,v) or (v,u) appears consecutively in p2 THEN append (u,v) to common_edges
// Step 2: Build subpaths from common edges
subpaths ← empty list
used ← empty set
FOR each edge (u,v) in common_edges DO
IF u,v not in used THEN create new subpath [u,v] and add u,v to used
ELSE IF u not in used THEN extend subpath containing v with u and add u to used
ELSE IF v not in used THEN extend subpath containing u with v and add v to used
// Step 2b: Add single-node subpaths for common nodes not in any edge
FOR each node in common_nodes DO
IF node not in used THEN append [node] as new subpath and add node to used
// Step 3: Add random nodes to reach half of total nodes
remaining_nodes ← nodes not in used
shuffle remaining_nodes randomly
target_size ← floor(total nodes / 2)
count_to_add ← max(0, target_size - size of used)
FOR i = 0 TO min(count_to_add-1, length of remaining_nodes-1) DO
append [remaining_nodes[i]] as new subpath and add to used
// Step 4: Randomly merge subpaths into offspring
shuffle subpaths randomly
offspring ← empty list
FOR each subpath sp in subpaths DO
IF offspring not empty AND random_boolean() THEN append reversed(sp) to offspring
ELSE append sp to offspring
RETURN offspring

3 Plots

After trying this algorithm, we will check its effectiveness by showing the constructed cycle.

In the plot, the cost of each node is represented by its size: larger nodes indicate a higher cost, while smaller ones indicate a lower cost. Additionally, all possible nodes are displayed, but the nodes chosen for the path are shown in blue, with their IDs printed on them. The started node is pointed with red color.

Figures for Steepest Local Search for Random Solution

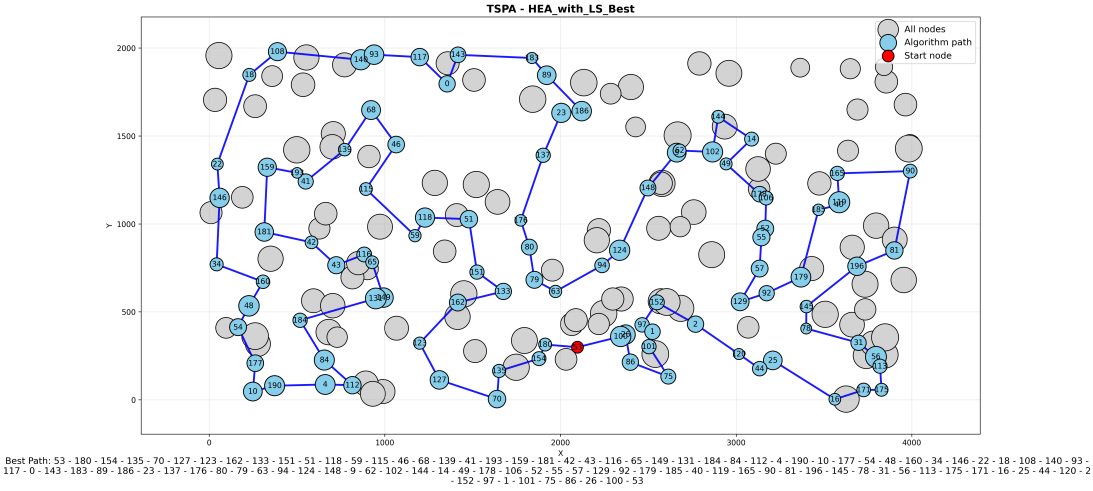


Figure 1: HEA with LS TSPA

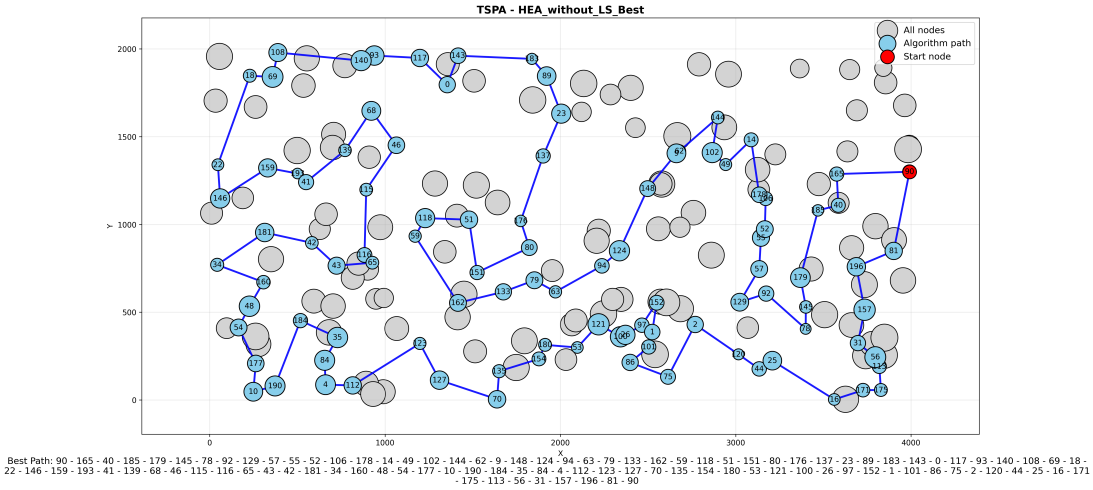
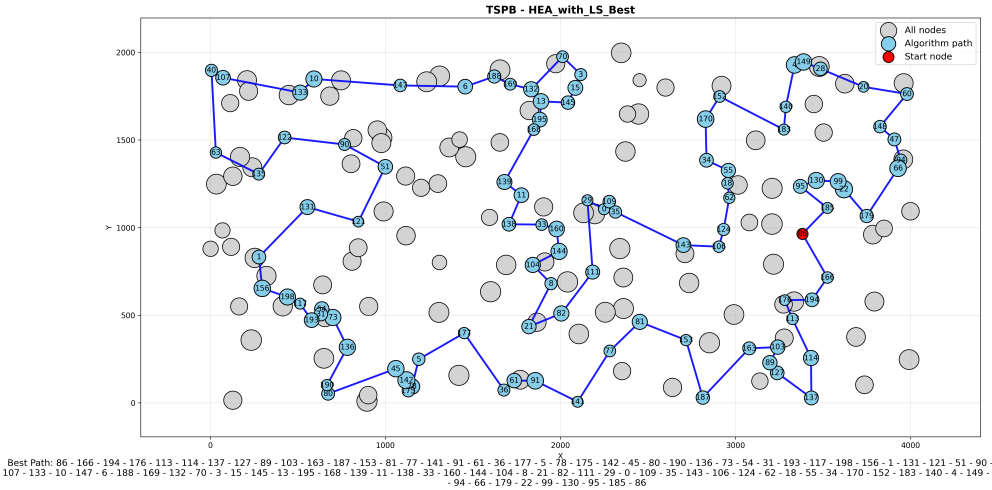
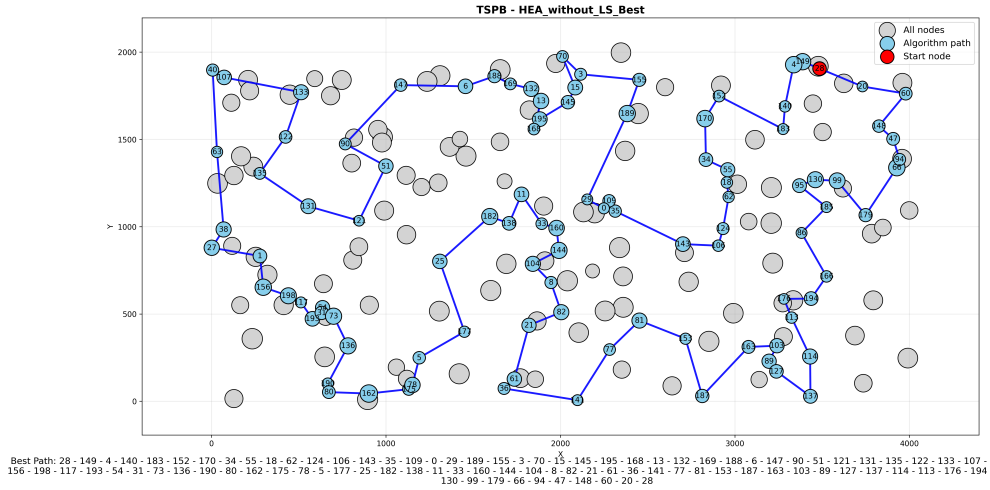


Figure 2: HEA without LS TSPA



Best Path: 86 - 166 - 194 - 176 - 113 - 114 - 137 - 127 - 89 - 103 - 163 - 187 - 153 - 81 - 77 - 141 - 91 - 61 - 36 - 177 - 5 - 78 - 175 - 142 - 45 - 80 - 190 - 136 - 73 - 54 - 31 - 193 - 117 - 198 - 156 - 1 - 131 - 121 - 51 - 90 - 122 - 135 - 63 - 40 - 107 - 133 - 10 - 147 - 6 - 188 - 169 - 132 - 70 - 3 - 15 - 145 - 13 - 195 - 168 - 139 - 11 - 138 - 33 - 160 - 144 - 104 - 8 - 21 - 82 - 111 - 29 - 0 - 109 - 35 - 143 - 106 - 124 - 62 - 18 - 55 - 34 - 170 - 152 - 183 - 140 - 4 - 149 - 28 - 20 - 60 - 148 - 47 - 94 - 66 - 179 - 22 - 99 - 130 - 95 - 185 - 86

Figure 3: HEA with LS TSPB



Best Path: 28 - 149 - 4 - 140 - 183 - 152 - 170 - 34 - 55 - 18 - 62 - 124 - 106 - 143 - 35 - 109 - 0 - 29 - 189 - 155 - 3 - 70 - 15 - 145 - 195 - 168 - 13 - 132 - 169 - 188 - 6 - 147 - 90 - 51 - 121 - 131 - 135 - 122 - 133 - 107 - 40 - 63 - 38 - 27 - 1 - 156 - 198 - 117 - 193 - 54 - 31 - 73 - 136 - 190 - 80 - 162 - 175 - 78 - 5 - 177 - 25 - 182 - 138 - 11 - 33 - 160 - 144 - 104 - 8 - 82 - 21 - 61 - 36 - 141 - 77 - 81 - 153 - 187 - 163 - 103 - 89 - 127 - 137 - 114 - 113 - 176 - 194 - 166 - 86 - 185 - 95 - 130 - 99 - 179 - 66 - 94 - 47 - 148 - 60 - 20 - 28

Figure 4: HEA without LS TSPB

4 Results

Method	TSPA (Avg, Min, Max)	TSPB (Avg, Min, Max)
Random Path	265530.42 (229719, 294157)	213673.99 (195590, 234518)
Local Steepest Edge (Random Path)	74200.79 (71517, 77754)	48421.32 (45631, 52371)
Local Steepest Edge with LM (Random Path)	74123.34 (71525, 78562)	49044.82 (46069, 52197)
MSLS (20 runs x 200 iterations)	71365.90 (70485, 71843)	45905.95 (45389, 46366)
ILS (20 runs, time-limited)	69627.05 (69141, 70453)	**43908.50 (43497, 44459)**
LNS (20 runs, time-limited, True)	**69543.00 (69207, 69904)**	43996.75 (43581, 44260)
LNS (20 runs, time-limited, False)	69746.00 (69263, 70633)	44256.45 (43671, 45105)
HEA (20 runs, time-limited, True)	**69203.55 (69107, 69383)**	**43598.45 (43446, 44024)**
HEA (20 runs, time-limited, False)	70302.25 (69722, 70901)	44619.30 (44026, 45200)

Table 1: Comparison of TSPA and TSPB results (Avg, Min, Max)

Method	TSPA Time (avg, min, max) [µs]	TSPB Time (avg, min, max) [µs]
Random Path	0.0036 (0.0033, 0.0176)	0.0052 (0.0044, 0.0239)
Local Steepest Edge (Random Path)	9.7100 (8.1613, 13.9574)	10.2649 (8.0421, 15.3924)
Local Steepest Edge with LM (Random Path)	9.3588 (6.6432, 12.3063)	9.3000 (7.0456, 12.0311)
MSLS (20 runs x 200 iterations)	1706.4574 (1689.2280, 1791.0236)	1900.8621 (1791.4707, 2033.1554)
ILS (20 runs, time-limited)	1706.6590 (1706.4574, 1707.1652)	1901.0305 (1900.8718, 1901.2089)
LNS (20 runs, time-limited, True)	1709.7860 (1707.4118, 1711.7095)	1903.8503 (1901.1062, 1907.0236)
LNS (20 runs, time-limited, False)	1709.8836 (1706.8554, 1712.9383)	1903.7007 (1901.0532, 1906.1929)
HEA (20 runs, time-limited, True)	1634.6455 (1634.3685, 1636.0765)	1716.7377 (1716.3562, 1717.6336)
HEA (20 runs, time-limited, False)	1634.9237 (1634.3497, 1636.1286)	1716.7424 (1716.3415, 1717.4731)

Table 2: Average, minimum, and maximum execution times for TSPA and TSPB

Configuration	TSPA(avg, min, max)	TSPB (avg, min, max)
HEA with Local Search	2508.80 (1927, 3429)	2347.20 (1798, 2886)
HEA without Local Search	20.00 (20, 20)	20.00 (20, 20)

Table 3: Comparison of Local Search Count for TSPA and TSPB (average, minimum, maximum)

Configuration	TSPA (avg, min, max)	TSPB (avg, min, max)
HEA with Local Search	2488.75 (1907, 3409)	2327.20 (1778, 2866)
HEA without Local Search	3134.05 (2030, 5708)	2919.50 (1983, 4616)

Table 4: Comparison of Main Loop Iterations for TSPA and TSPB (average, minimum, maximum)

5 Conclusion

Based on the results in Tables 1, 2, the following observations can be made:

The Hybrid Evolutionary Algorithm (HEA) achieved best performance over other algorithms when **local search was enabled after recombination** (`useLocalSearch = true`), producing consistently better solutions. The **dual recombination strategy** was key: Operator 1 preserves common nodes and edges as subpaths while adding random nodes to maintain diversity, and Operator 2 starts from one parent, removes nodes absent in the other, and repairs the solution heuristically, preserving order and traversal directions. Combining these operators with post-recombination local search allowed the algorithm to explore the solution space efficiently, resulting in significantly improved solution quality and stability.

Interestingly, choosing to disable local search after recombination, even though it changes the objective value only slightly it makes a noticeable difference between algorithm performances. Nonetheless, other algorithms such as LNS and ILS still achieve comparable results in both TSP problems.

6 All Other Results

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Path	264188.61 (238257, 293017)	213973.45 (184171, 238914)
Nearest Neighbor (End)	85108.51 (83182, 89433)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)	48694.25 (44609, 57283)
Greedy Cycle	72635.98 (71488, 74410)	51400.60 (49001, 57324)
Greedy 2-Regret	115400.24 (105692, 126860)	72712.85 (67809, 78406)
Greedy 2-Regret (w=0.5)	72136.15 (71108, 73395)	50934.60 (47144, 55700)
NN-Flex 2-Regret	117138.49 (108151, 124921)	74444.46 (69933, 80278)
NN-Flex 2-Regret (w=0.5)	72407.59 (70010, 75452)	47664.46 (44891, 55247)
Local Greedy Node (Greedy Cycle)	70687.00 (70687, 70687)	46443.67 (46328, 51512)
Local Greedy Edge (Greedy Cycle)	70618.84 (70571, 70663)	46089.89 (45824, 51482)
Local Steepest Node (Greedy Cycle)	70687.00 (70687, 70687)	46466.54 (46371, 51512)
Local Steepest Edge (Greedy Cycle)	70571.00 (70571, 70571)	45996.84 (45867, 51259)
Local Greedy Node (Random Path)	85217.85 (77570, 94277)	60178.87 (54006, 66885)
Local Greedy Edge (Random Path)	74232.03 (71777, 77750)	48192.19 (46109, 51934)
Local Steepest Node (Random Path)	85271.00 (85271, 85271)	63491.00 (63491, 63491)
Local Steepest Edge (Random Path)	72687.00 (72687, 72687)	49190.00 (49190, 49190)
Local Steepest (Candidates, k=5)	88563.00 (88563, 88563)	52093.00 (52093, 52093)
Local Steepest (Candidates, k=10)	83598.00 (83598, 83598)	51153.00 (51153, 51153)
Local Steepest(Candidates, k=15)	81136.00 (81136, 81136)	48854.00 (48854, 48854)
Local Steepest Edge with LM (Random Path)	74187.41 (71646, 78764)	49144.47 (45888, 53039)

Table 5: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The 3 lowest average values in each column are highlighted in bold.

Method	TSPA Time Avg (Min, Max)	TSPB Time Avg(Min, Max)
Random Path	0.0036 (0.0034, 0.0144)	0.0039 (0.0036, 0.0232)
Nearest Neighbor (End)	0.0694 (0.0463, 0.1093)	0.0685 (0.0470, 0.1282)
Nearest Neighbor (Flexible)	91.0993 (85.8761, 119.1184)	94.7285 (87.4900, 154.2259)
Greedy Cycle	87.6467 (82.1853, 135.8448)	90.6419 (84.6333, 153.7690)
Greedy 2-Regret	13.4090 (12.2387, 18.2542)	13.4612 (12.4574, 21.9434)
Greedy 2-Regret (w=0.5)	12.9063 (11.8320, 18.2641)	12.6926 (11.2819, 20.7204)
NN-Flex 2-Regret	13.8750 (12.7669, 18.4833)	13.9626 (12.9940, 21.1062)
NN-Flex 2-Regret (w=0.5)	13.5540 (12.4458, 19.2731)	13.6345 (12.6257, 21.8523)
Local Greedy Node (Greedy Cycle*)	1.2763 (0.8682, 2.4062)	1.8935 (1.1700, 5.2516)
Local Greedy Edge (Greedy Cycle*)	1.5676 (1.1047, 3.2752)	2.5081 (1.3700, 5.8678)
Local Steepest Node (Greedy Cycle*)	0.5265 (0.4719, 1.0174)	0.7846 (0.4909, 1.6765)
Local Steepest Edge (Greedy Cycle*)	0.5418 (0.4837, 1.0700)	1.0705 (0.6829, 2.2377)
Local Greedy Node (Random Path)	57.7227 (44.0444, 123.7979)	51.7444 (41.5894, 92.8947)
Local Greedy Edge (Random Path)	58.0691 (47.6349, 99.7285)	52.4021 (45.1037, 84.4763)
Local Steepest Node (Random Path)	10.9774 (10.0351, 18.7647)	13.6565 (12.4567, 26.2057)
Local Steepest Edge (Random Path)	9.1549 (8.5878, 12.7278)	9.0840 (8.4419, 18.6255)
Local Steepest (Candidates, k=5)	1.4490 (1.3910, 1.9627)	1.4250 (1.3467, 1.8863)
Local Steepest (Candidates, k=10)	1.8156 (1.7346, 2.3018)	1.7992 (1.7277, 2.4961)
Local Steepest (Candidates, k=15)	2.1838 (2.0927, 2.7651)	2.1374 (2.0609, 2.9937)
Local Steepest Edge with LM (Random Path)	50.6919 (35.5602, 69.0622)	53.7797 (39.3278, 92.4221)

Table 6: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)